# LAB Exam

**NAME:** Hajra Sarwar
**REG NO:** 2023-BSE-022
**CLASS:** BSE-5A
**SUBJECT:** CC

## QUESTION1:

1. **Create Group**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

@hajrasarwar11 →/workspaces/Lab_Exam (main) $ aws iam create-group --group-name SoftwareEngineering
{
    "Group": {
        "Path": "/",
        "GroupName": "SoftwareEngineering",
        "GroupId": "AGPA5WPGPNRITFPWXUVOL",
        "Arn": "arn:aws:iam::941618064465:group/SoftwareEngineering",
        "CreateDate": "2026-01-19T07:32:07+00:00"
    }
}
```

**Group Details:**

```
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ aws iam get-group --group-name SoftwareEngineering
{
    "Users": [],
    "Group": {
        "Path": "/",
        "GroupName": "SoftwareEngineering",
        "GroupId": "AGPA5WPGPNRITFPWXUVOL",
        "Arn": "arn:aws:iam::941618064465:group/SoftwareEngineering",
        "CreateDate": "2026-01-19T07:32:07+00:00"
    }
}
@hajrasarwar11 →/workspaces/Lab_Exam (main) $
```

2. **Iam User:**

```
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ aws iam create-user --user-name Hajra
{
    "User": {
        "Path": "/",
        "UserName": "Hajra",
        "UserId": "AIDA5WPGPNRIZ6WVBH2DR",
        "Arn": "arn:aws:iam::941618064465:user/Hajra",
        "CreateDate": "2026-01-19T07:36:27+00:00"
    }
}
@hajrasarwar11 →/workspaces/Lab_Exam (main) $
```

**User Detail:**

```
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ aws iam get-user --user-name Hajra
{
    "User": {
        "Path": "/",
        "UserName": "Hajra",
        "UserId": "AIDA5WPGPNRIZ6WVBH2DR",
        "Arn": "arn:aws:iam::941618064465:user/Hajra",
        "CreateDate": "2026-01-19T07:36:27+00:00"
    }
}
@hajrasarwar11 →/workspaces/Lab_Exam (main) $
```

3. **Add user to group:**

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ aws iam add-user-to-group --user-name Hajra --group-name SoftwareEngineering
```

**Group Detail:**

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ aws iam get-group --group-name SoftwareEngineering
{
    "Users": [
        {
            "Path": "/",
            "UserName": "Hajra",
            "UserId": "AIDA5WPGPNRIZ6WVBH2DR",
            "Arn": "arn:aws:iam::941618064465:user/Hajra",
            "CreateDate": "2026-01-19T07:36:27+00:00"
        }
    ],
    "Group": {
        "Path": "/",
        "GroupName": "SoftwareEngineering",
        "GroupId": "AGPA5WPGPNRITFPWXUVOL",
        "Arn": "arn:aws:iam::941618064465:group/SoftwareEngineering",
        "CreateDate": "2026-01-19T07:32:07+00:00"
    }
}
```

4. **Attach Administrator access policy:**

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ aws iam list-policies \
> --query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
> --output text
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
AmazonEC2RoleforAWSCodeDeploy
AmazonEC2RoleforSSM
CloudWatchActionsEC2Access
AmazonEC2ContainerRegistryReadOnly
AmazonEC2ContainerRegistryPowerUser
AmazonEC2ContainerRegistryFullAccess
AmazonEC2ContainerServiceAutoscaleRole
AmazonEC2SpotFleetAutoscaleRole
AWSElasticBeanstalkCustomPlatformforEC2Role
AmazonEC2ContainerServiceEventsRole
AmazonEC2SpotFleetTaggingRole
AWSEC2SpotServiceRolePolicy
AWSServiceRoleForEC2ScheduledInstances
AWSEC2SpotFleetServiceRolePolicy
AWSApplicationAutoscalingEC2SpotFleetRequestPolicy
AWSEC2FleetServiceRolePolicy
AWSAutoScalingPlansEC2AutoScalingPolicy
:
```

**Administrator access policy with arn:**

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ aws iam list-policies --query 'Policies[?PolicyName==`AdministratorAccess`].{Name:PolicyName, ARN:Arn}' --output table
----------------------------------------------------------------------
|                          ListPolicies                              |
+-----------------------------------------------+--------------------+
|                      ARN                      |        Name        |
+-----------------------------------------------+--------------------+
|  arn:aws:iam::aws:policy/AdministratorAccess  |  AdministratorAccess  |
+-----------------------------------------------+--------------------+
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

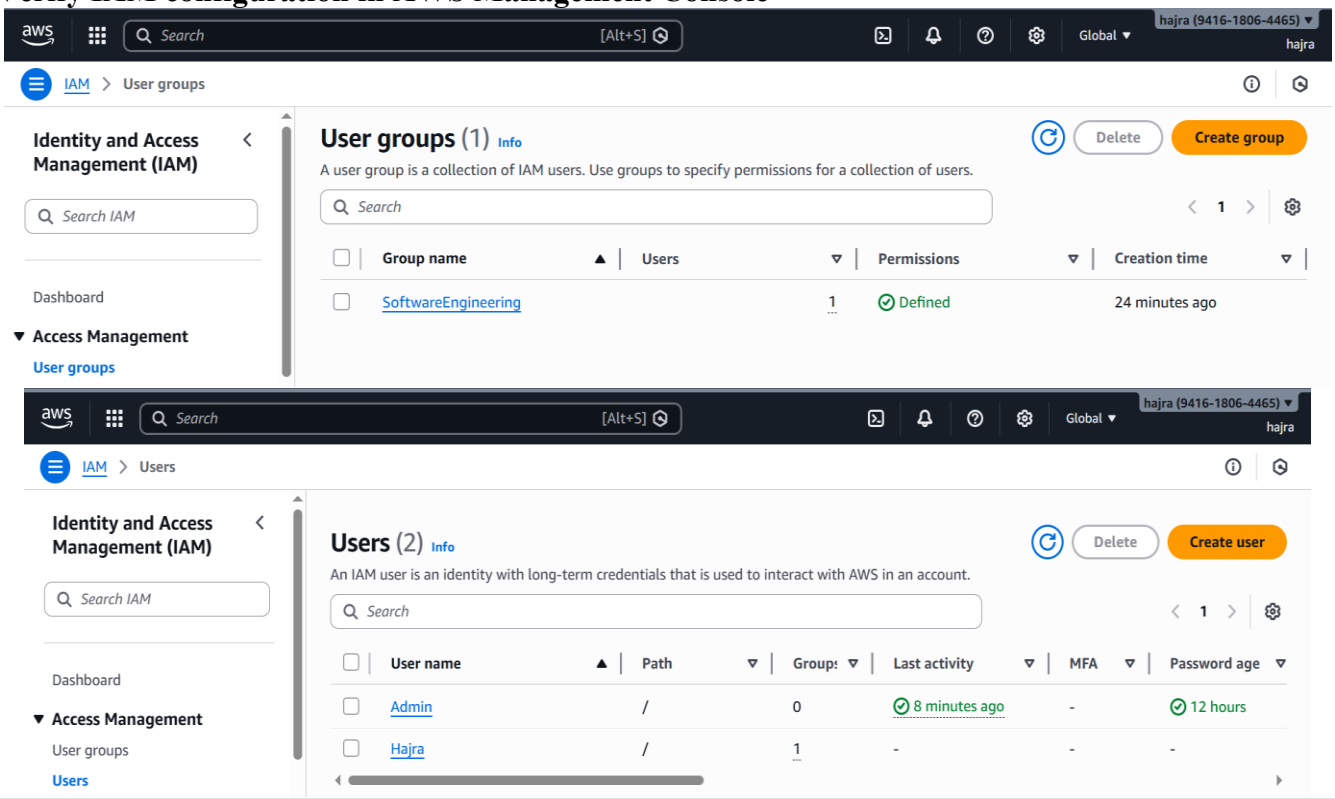
**Attach the AdministratorAccess policy to the group:**

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ aws iam attach-group-policy \
> --group-name SoftwareEngineering \
> --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

5. **List attach policies:**

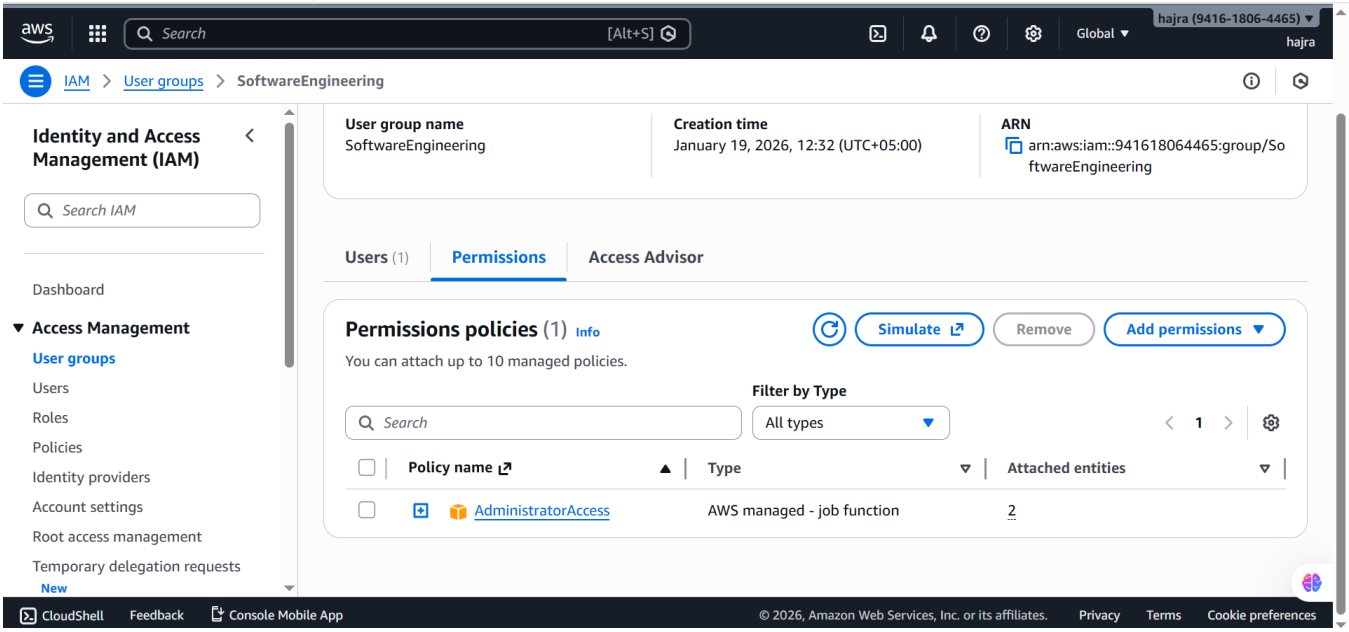
```
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ aws iam list-attached-group-policies --group-name SoftwareEngineering
{
    "AttachedPolicies": [
        {
            "PolicyName": "AdministratorAccess",
            "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
        }
    ]
}
@hajrasarwar11 →/workspaces/Lab_Exam (main) $
main*  ⊗ 0 ⚠ 0  0  ✓ AWS: profile:default
```

6. **Verify IAM configuration in AWS Management Console**







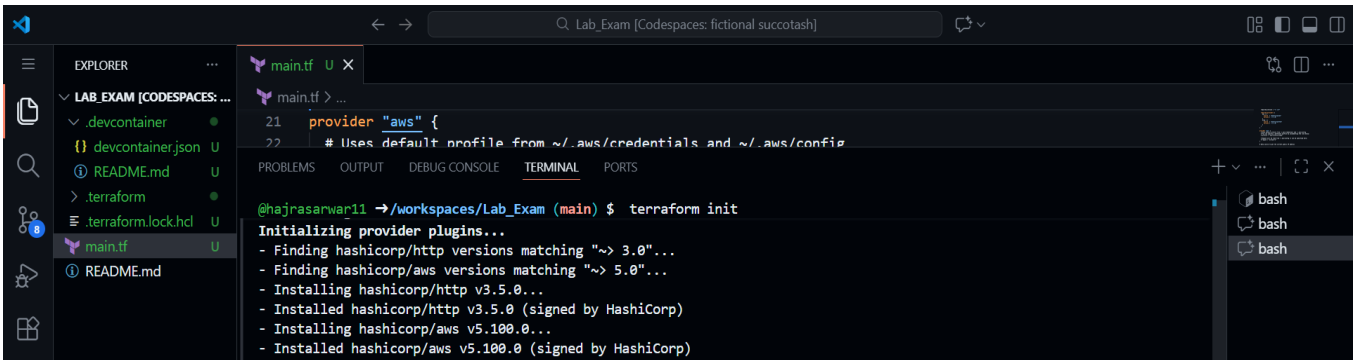


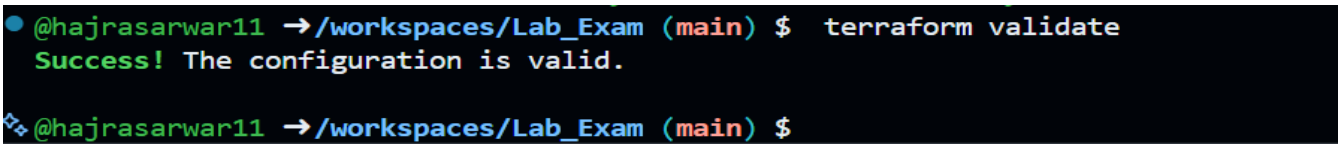
**QUESTION 2:**

**1. Configure the AWS provider**

**CODE:**

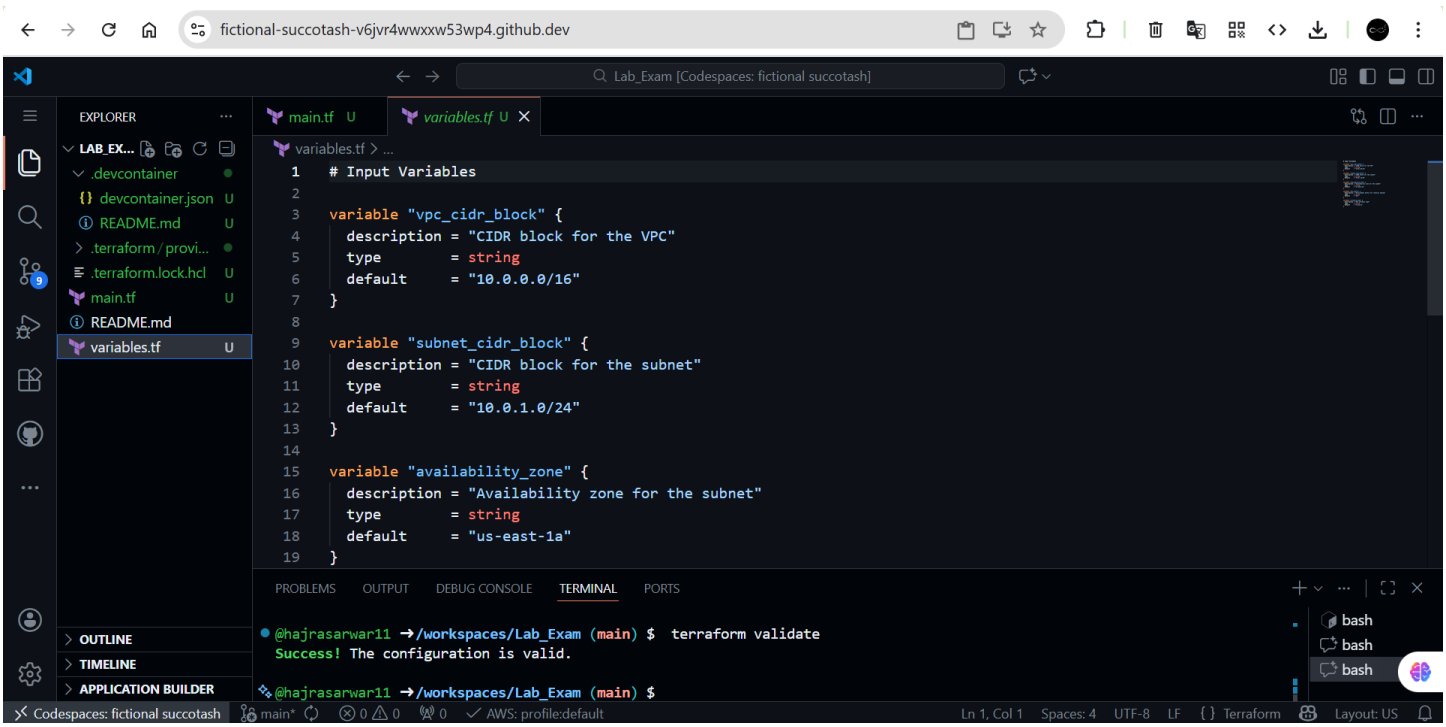
```
# Provider Configuration
# The AWS provider will read credentials from:
# - ~/.aws/credentials (for access keys)
# - ~/.aws/config (for region and other settings)

terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    http = {
      source  = "hashicorp/http"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  # Uses default profile from ~/.aws/credentials and ~/.aws/config
  # You can specify a different profile by uncommenting the line below:
  # profile = "your-profile-name"

  # Region will be read from ~/.aws/config or can be specified here:
  # region = "us-east-1"
}

# Data source to get the current public IP address
data "http" "my_ip" {
  url = "https://ifconfig.me/ip"
}

# Locals block to format the IP address with /32 CIDR notation
locals {
  my_ip_cidr = "${chomp(data.http.my_ip.response_body)}/32"
}
```




2. **Define input variables:**
   **CODE:**

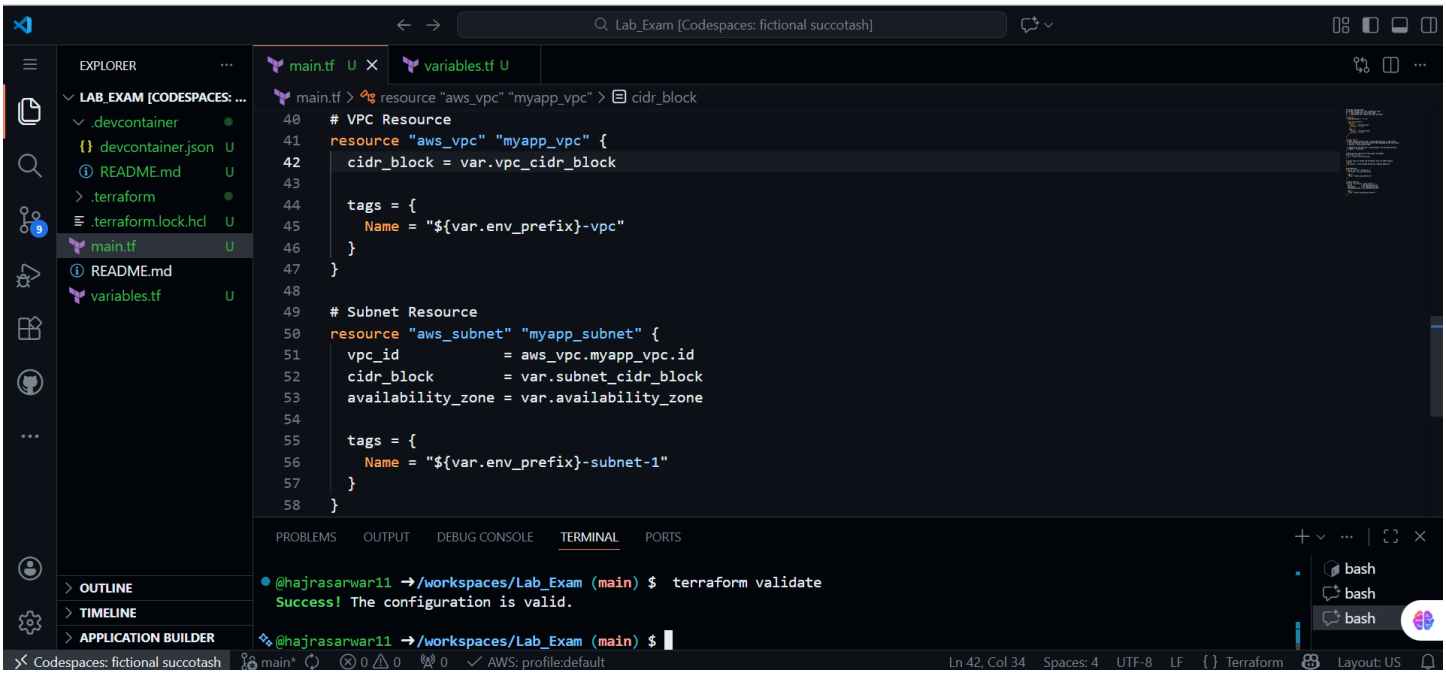
```
# Input Variables

variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "subnet_cidr_block" {
  description = "CIDR block for the subnet"
  type        = string
  default     = "10.0.1.0/24"
}

variable "availability_zone" {
  description = "Availability zone for the subnet"
  type        = string
  default     = "us-east-1a"
}

variable "env_prefix" {
  description = "Environment prefix for resource naming"
  type        = string
  default     = "dev"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```




### 3. Create VPC and subnet:

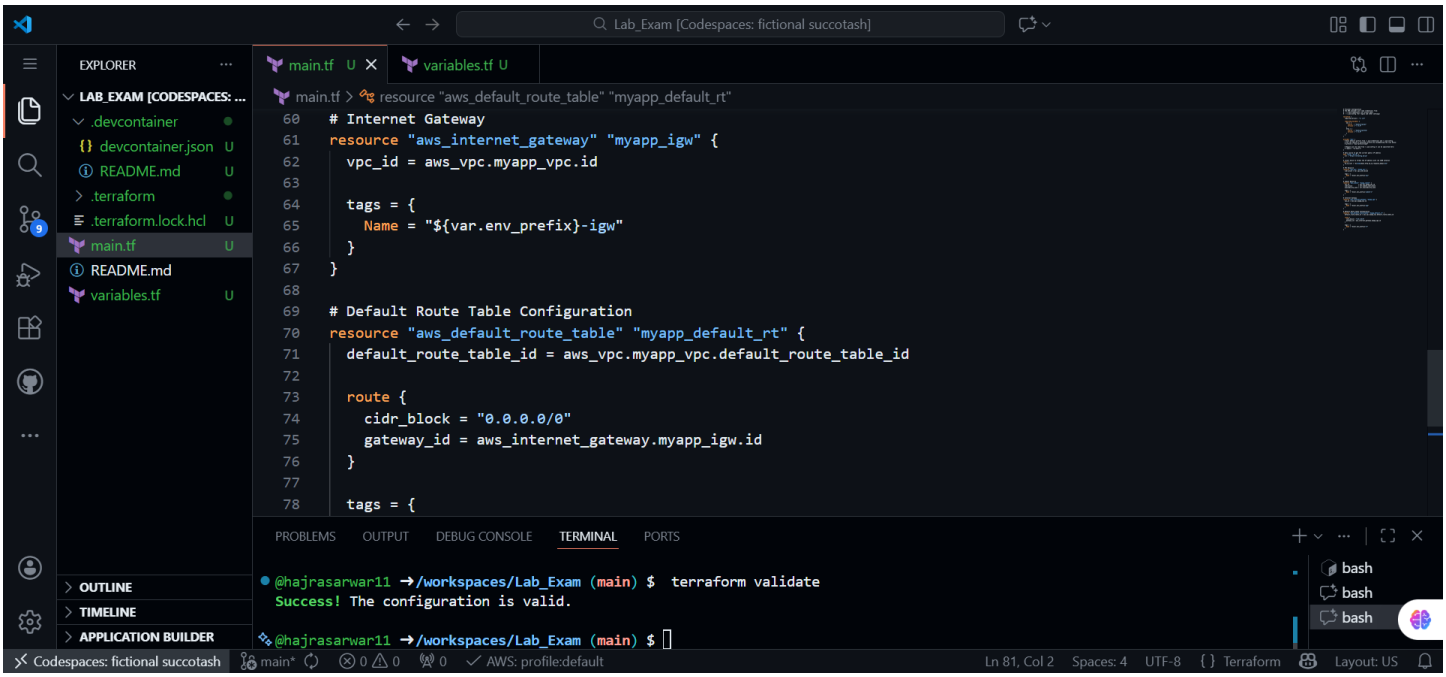
**CODE:**

```
# VPC Resource
resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

# Subnet Resource
resource "aws_subnet" "myapp_subnet" {
  vpc_id            = aws_vpc.myapp_vpc.id
  cidr_block        = var.subnet_cidr_block
  availability_zone = var.availability_zone

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
```




**4. Internet Gateway and configure route table:**
**CODE:**

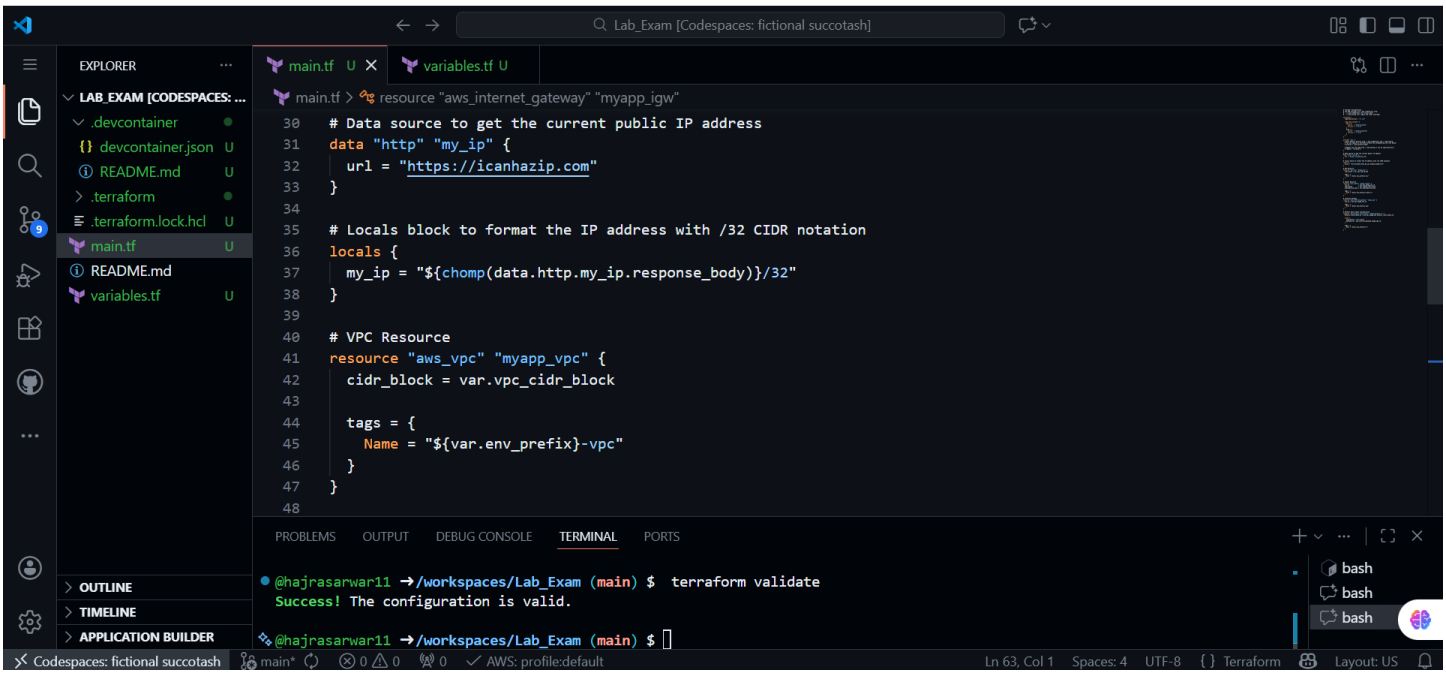
```
# Internet Gateway
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

# Default Route Table Configuration
resource "aws_default_route_table" "myapp_default_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
```




## 5. Discover IP

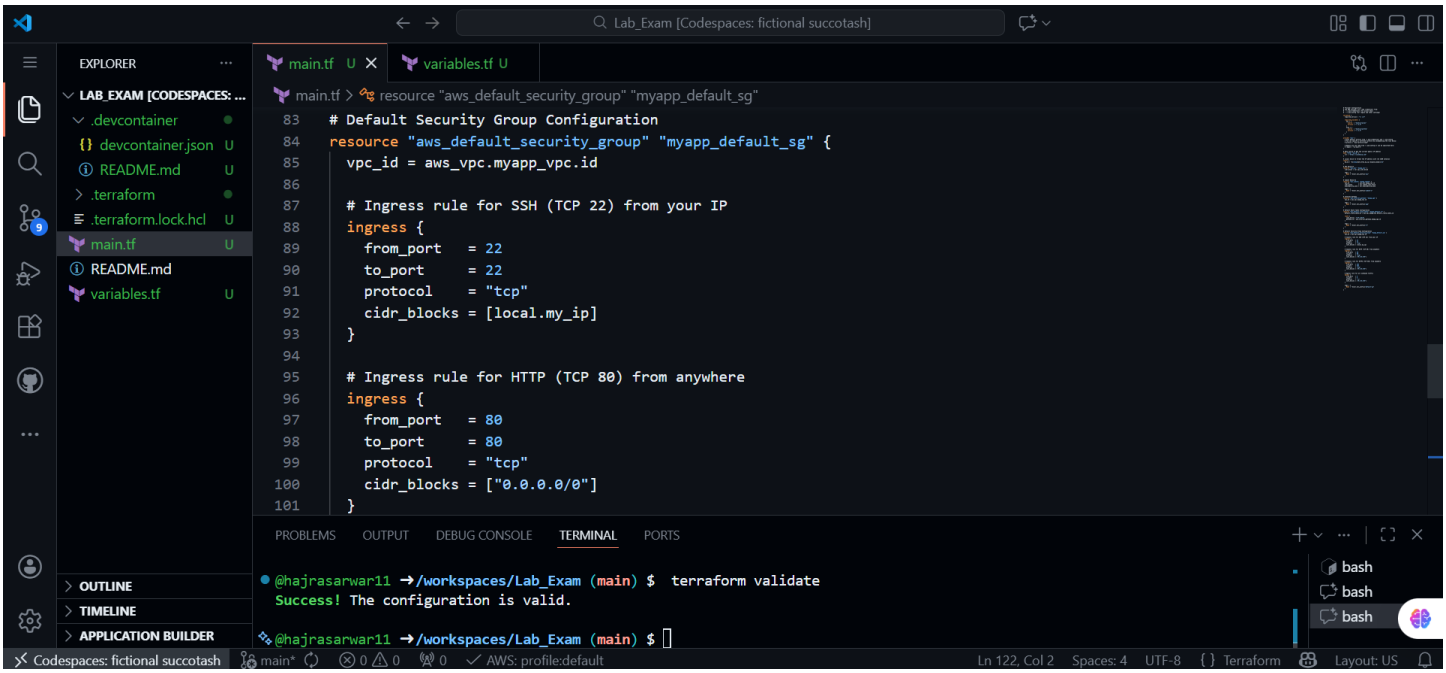
**CODE:**

```
# Data source to get the current public IP address
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

# Locals block to format the IP address with /32 CIDR notation
locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"
}
```



6. **Configue the default security group:**
   **CODE:**

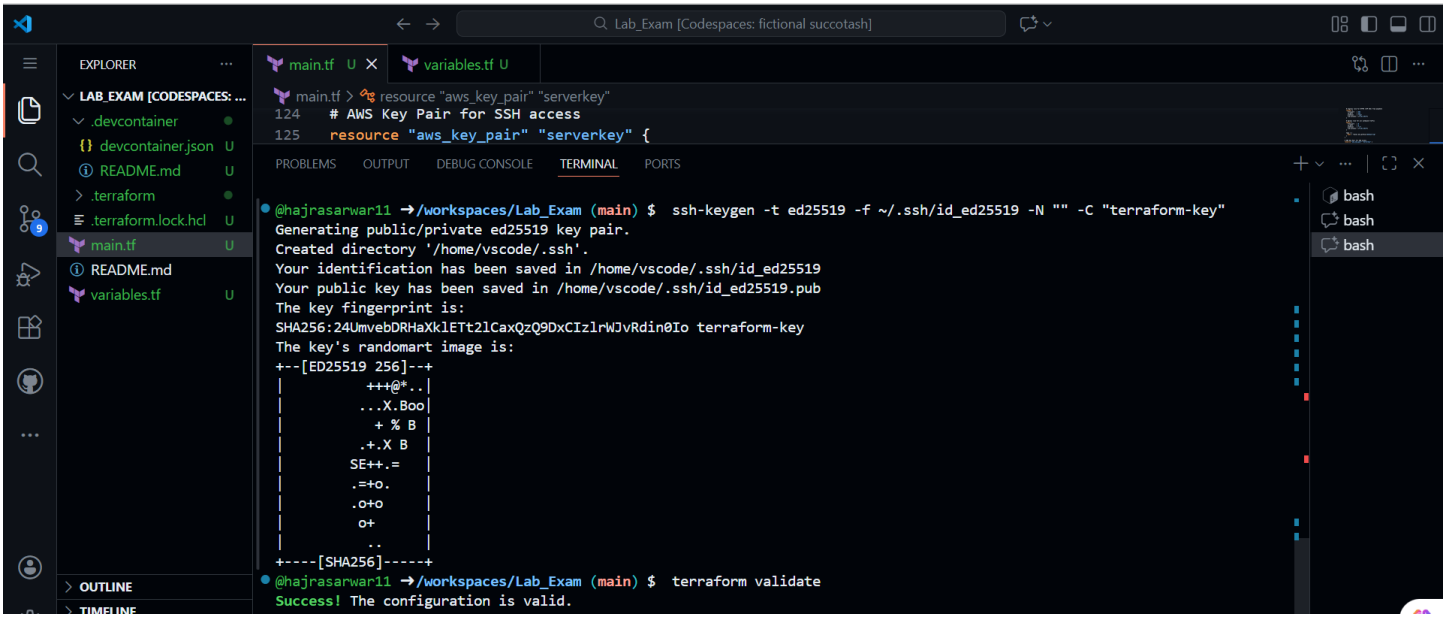
```
# Default Security Group Configuration
resource "aws_default_security_group" "myapp_default_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  # Ingress rule for SSH (TCP 22) from your IP
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [local.my_ip]
  }

  # Ingress rule for HTTP (TCP 80) from anywhere
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Ingress rule for HTTPS (TCP 443) from anywhere
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Egress rule for all outbound traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.env_prefix}-default-sg"
  }
}
```



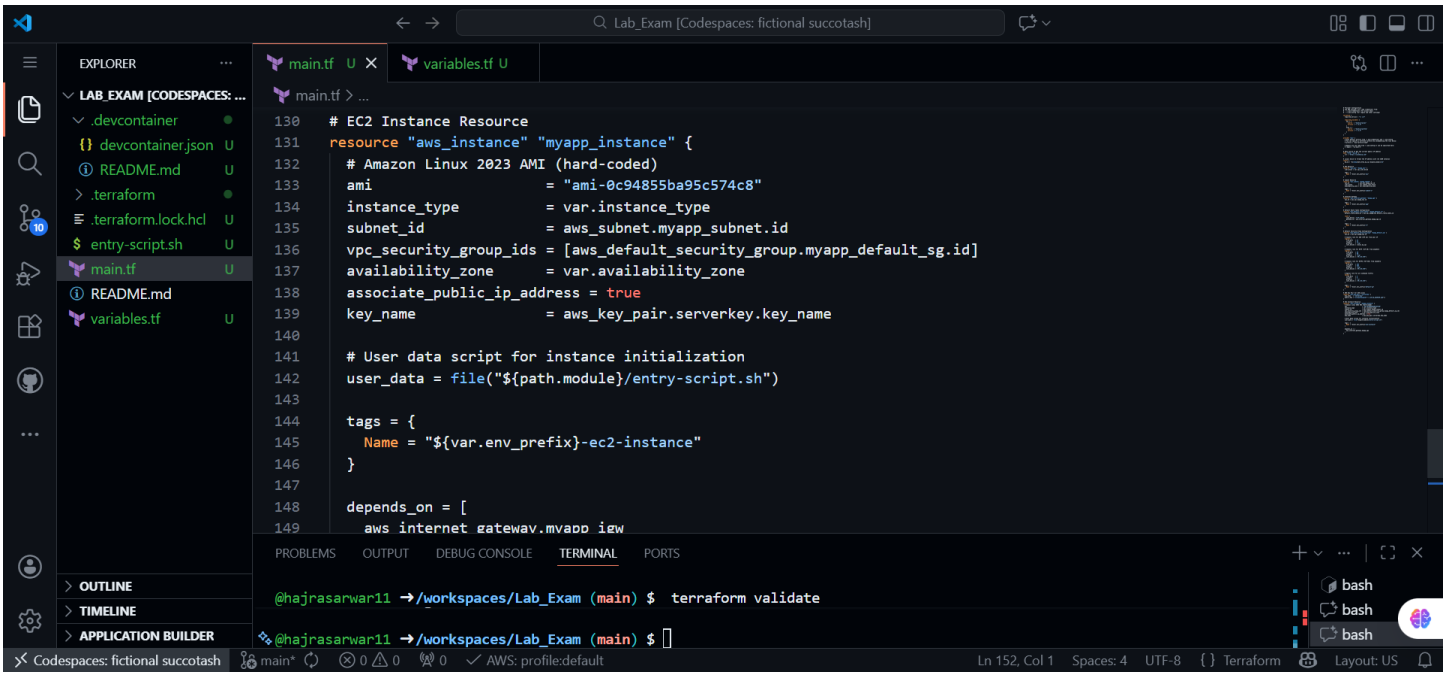

## 7. create an AWS key pair for SSH

**CODE:**

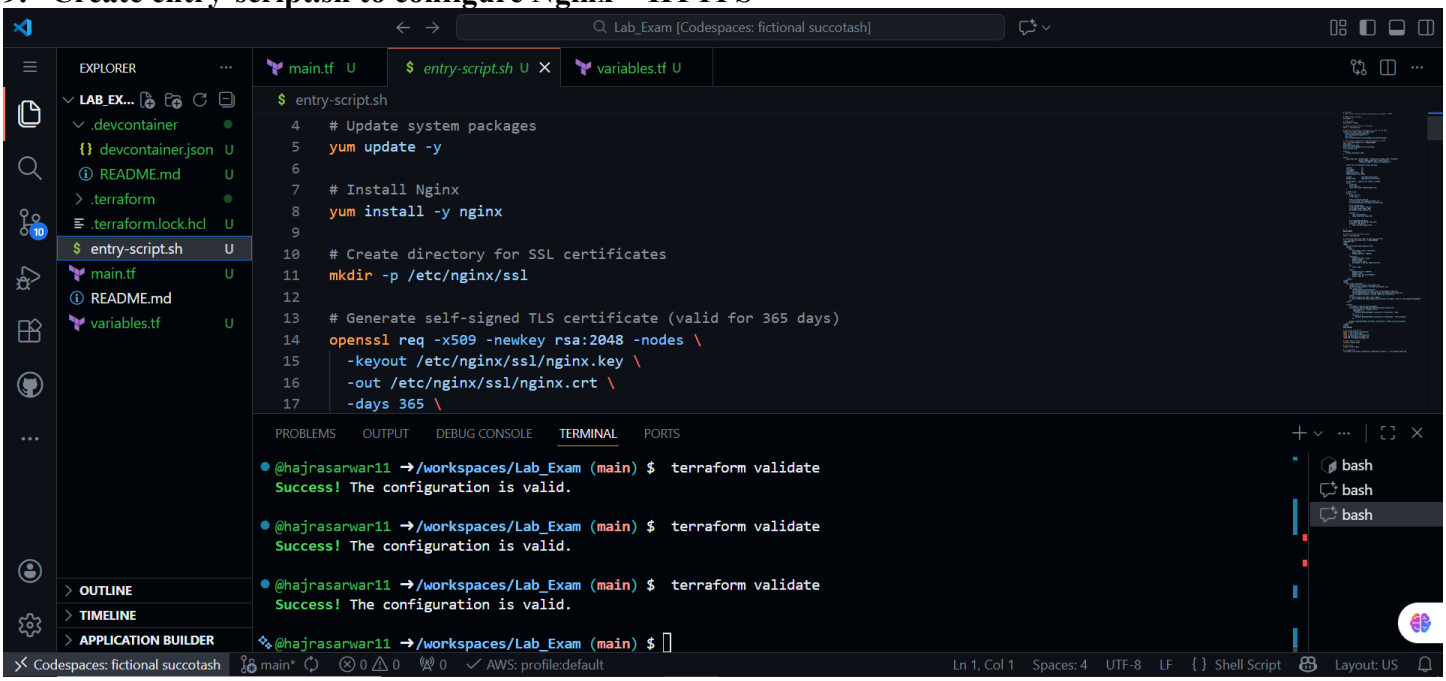
```
# AWS Key Pair for SSH access
resource "aws_key_pair" "serverkey" {
  key_name   = "serverkey"
  public_key = file(pathexpand("~/.ssh/id_ed25519.pub"))
}
```



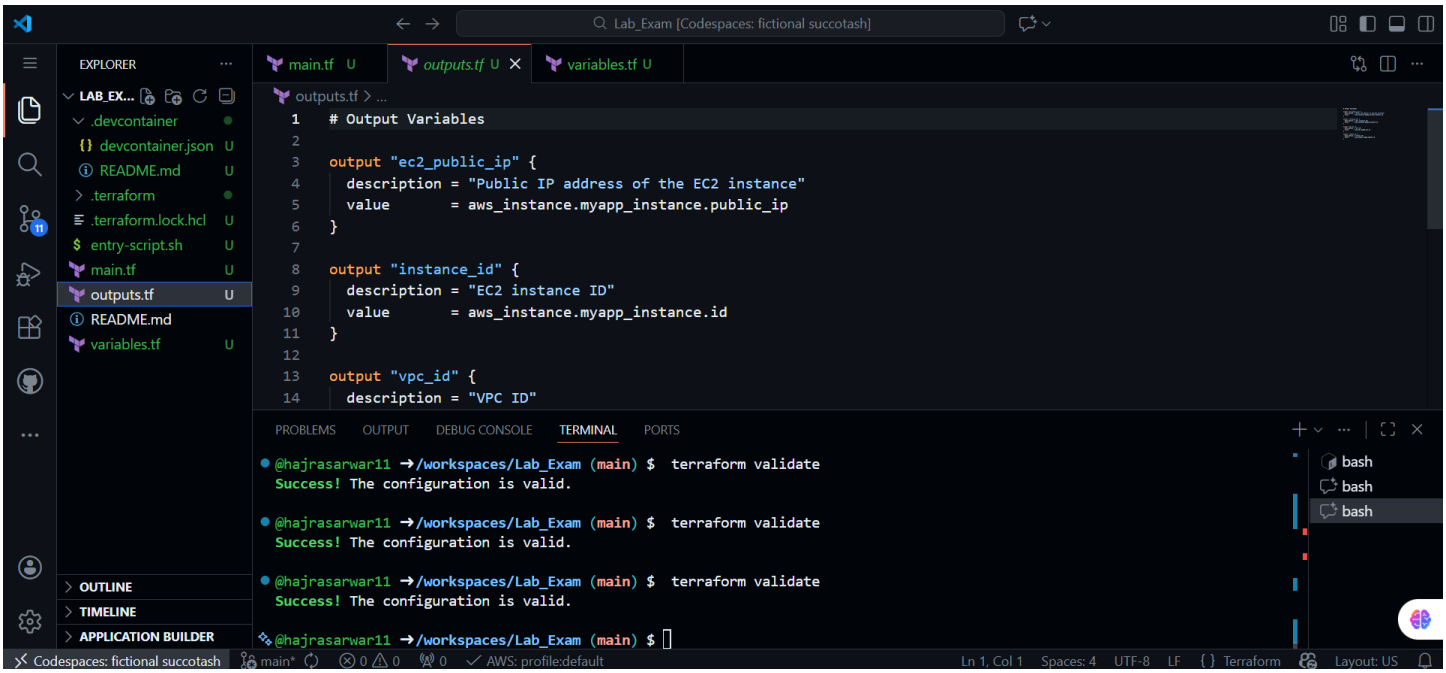

## 8. Create EC2 instance:



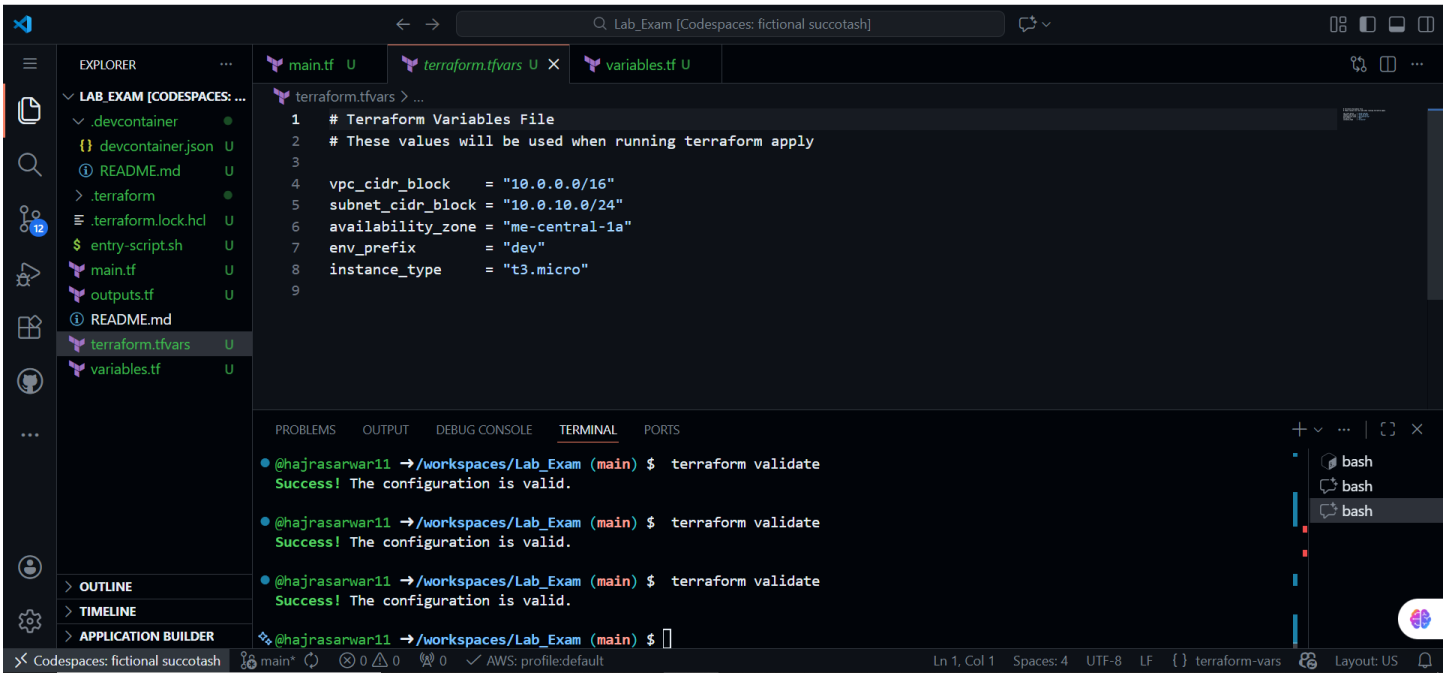

## 9. Create entry-script.sh to configure Nginx + HTTPS



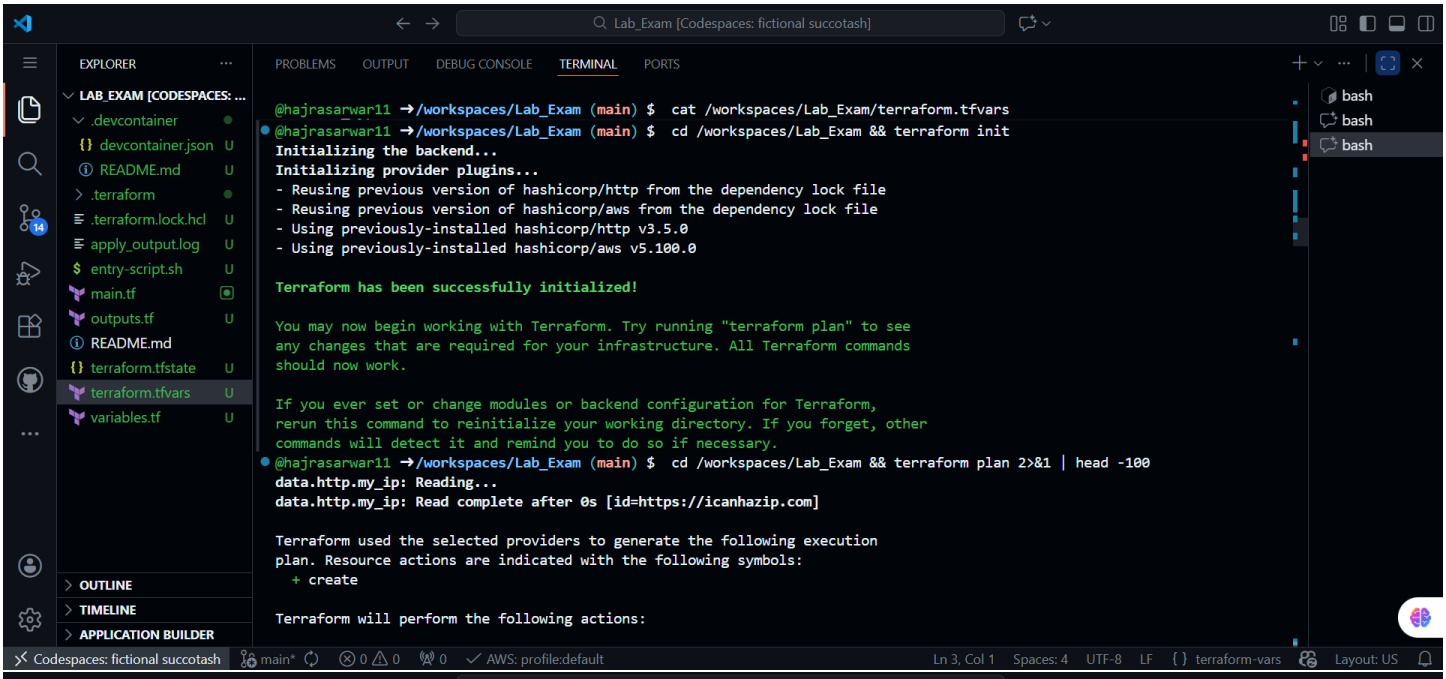

## 10. Add teraaform output:

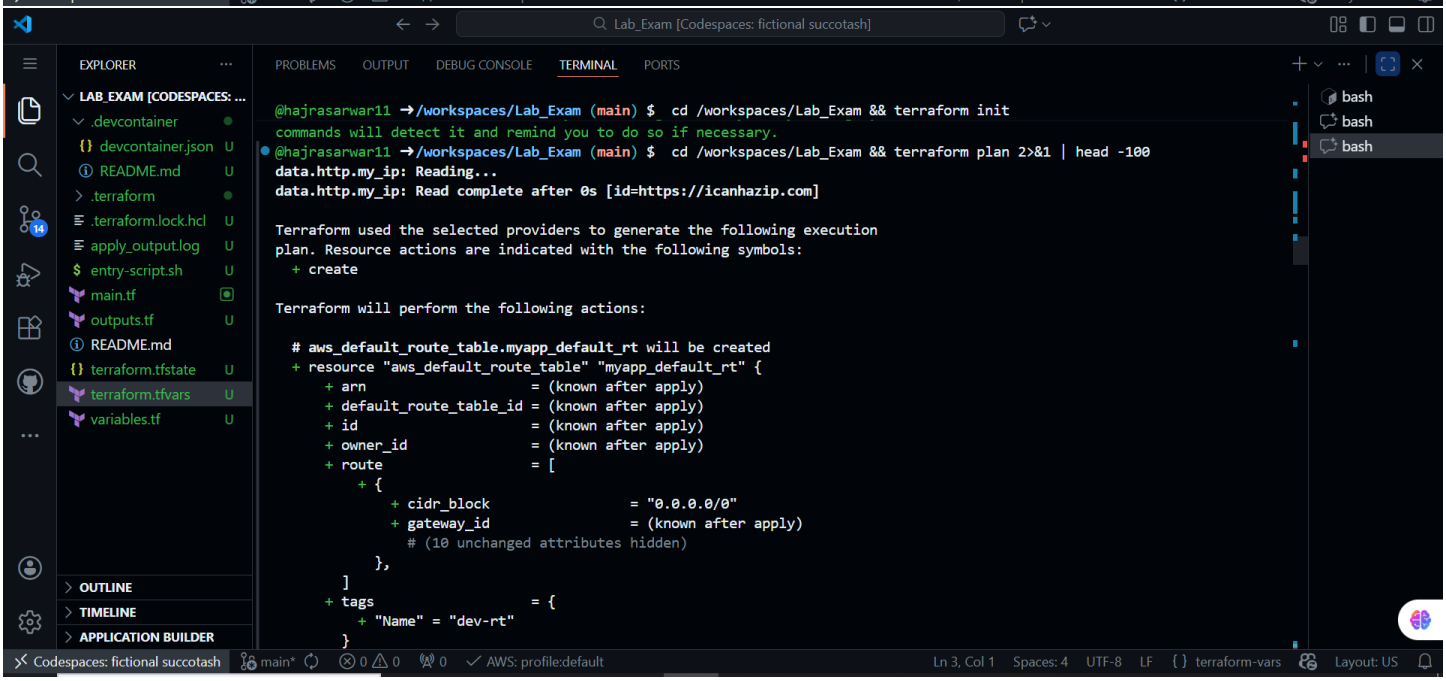


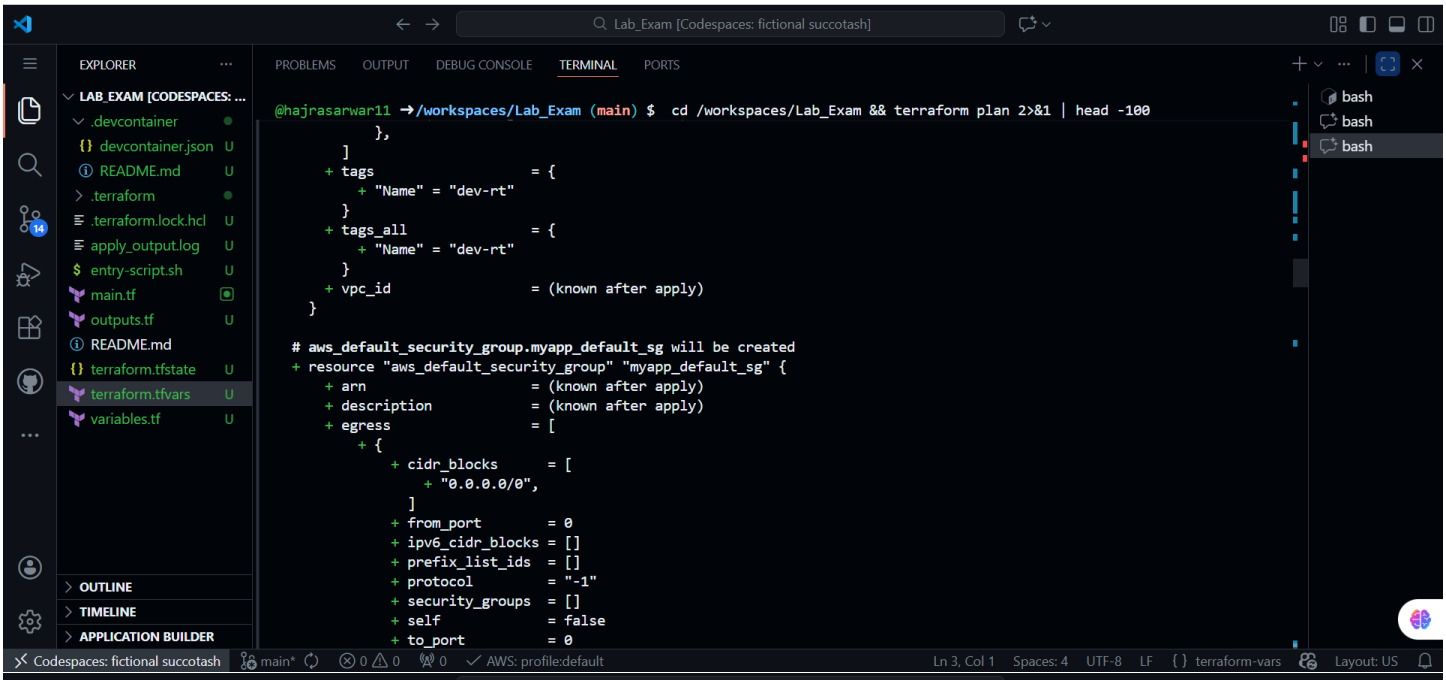

## 11. Set vararible values:

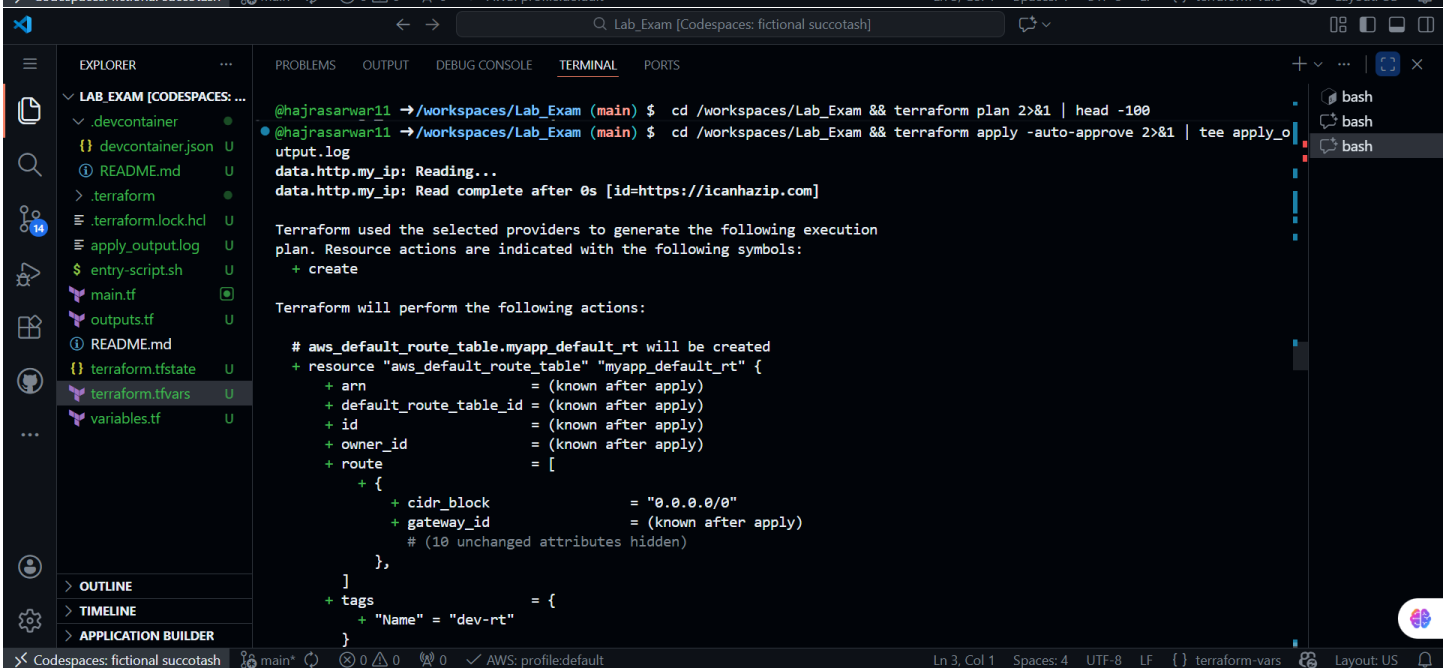


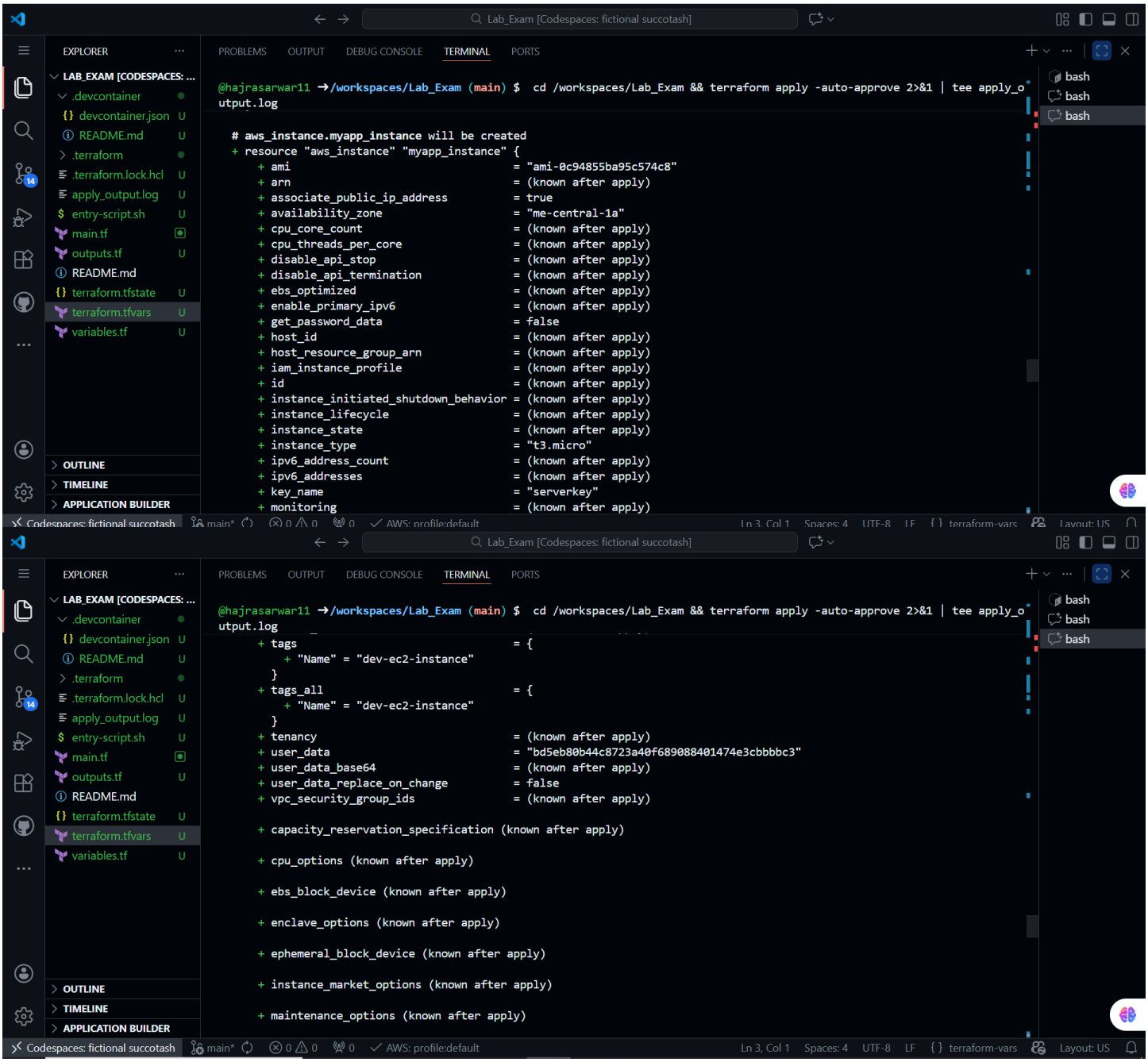

## 12. Commands:


Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json U
README.md U
.terraform
.terraform.lock.hcl U
apply_output.log U
entry-script.sh U
main.tf
outputs.tf U
README.md
terraform.tfstate U
terraform.tfvars U
variables.tf U
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cat /workspaces/Lab_Exam/terraform.tfvars
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/http v3.5.0
- Using previously-installed hashicorp/aws v5.100.0
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform plan 2>&1 | head -100
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
bash
bash
bash
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US

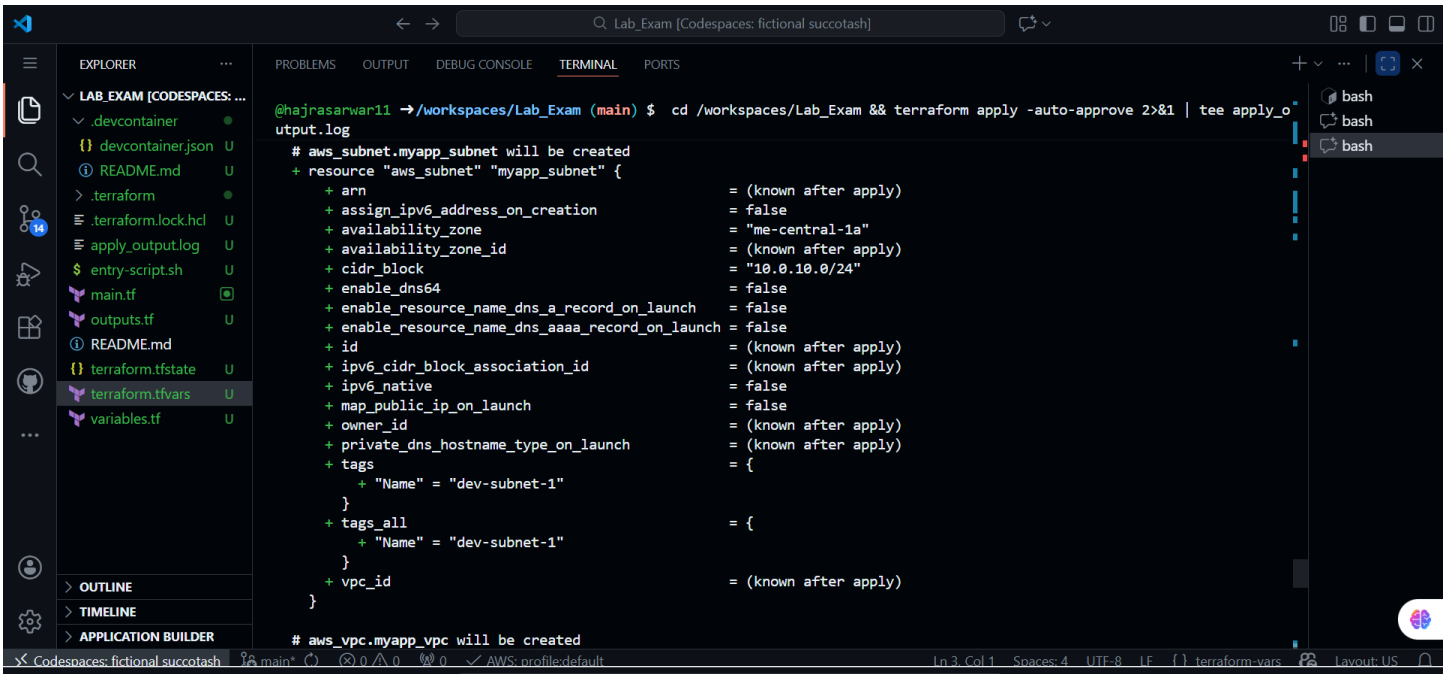

Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json U
README.md U
.terraform
.terraform.lock.hcl U
apply_output.log U
entry-script.sh U
main.tf
outputs.tf U
README.md
terraform.tfstate U
terraform.tfvars U
variables.tf U
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform init
commands will detect it and remind you to do so if necessary.
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform plan 2>&1 | head -100
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
# aws_default_route_table.myapp_default_rt will be created
+ resource "aws_default_route_table" "myapp_default_rt" {
+ arn = (known after apply)
+ default_route_table_id = (known after apply)
+ id = (known after apply)
+ owner_id = (known after apply)
+ route = [
+ {
+ cidr_block = "0.0.0.0/0"
+ gateway_id = (known after apply)
# (10 unchanged attributes hidden)
},
]
+ tags = {
+ "Name" = "dev-rt"
}
bash
bash
bash
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US

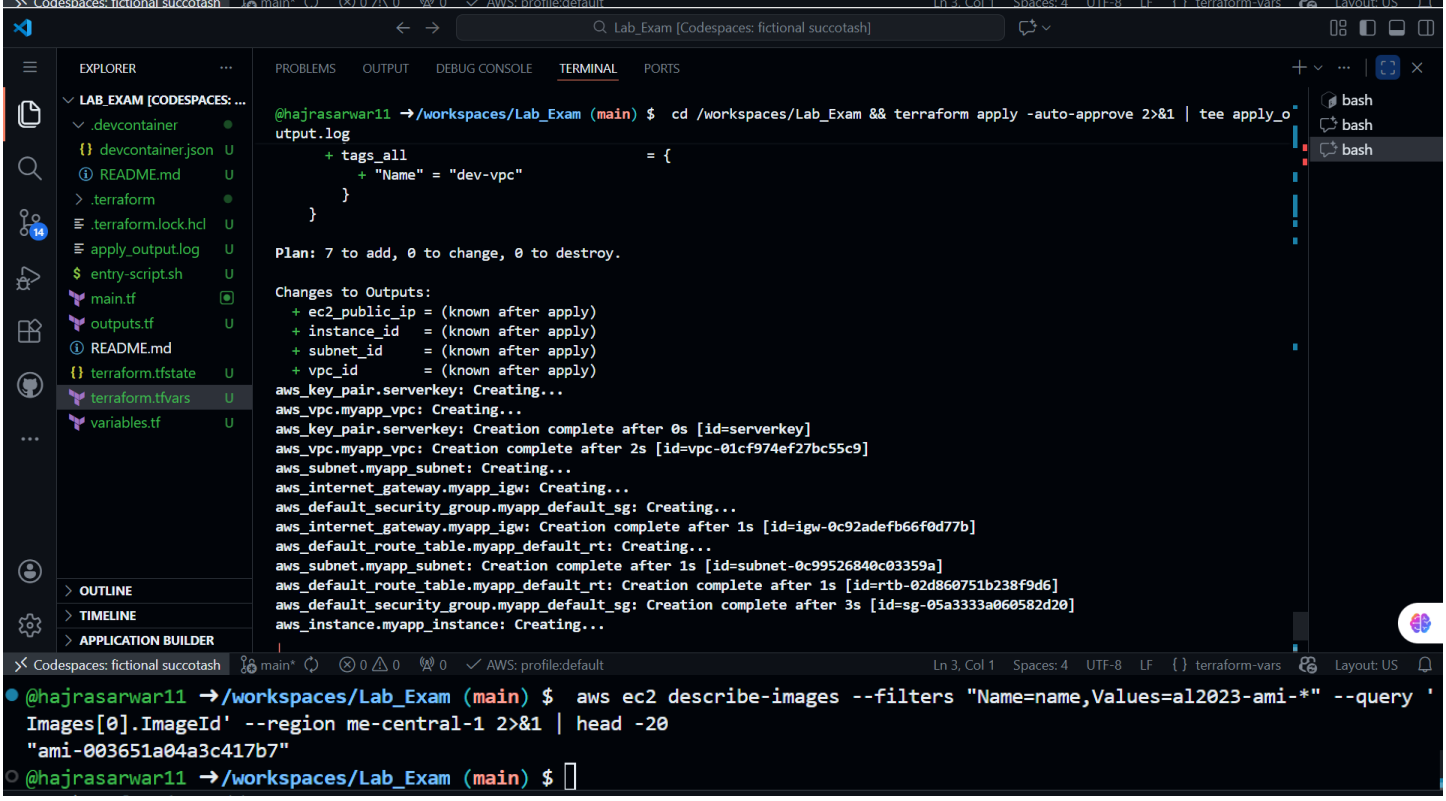

Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json U
README.md U
.terraform
.terraform.lock.hcl U
apply_output.log U
entry-script.sh U
main.tf
outputs.tf U
README.md
terraform.tfstate U
terraform.tfvars U
variables.tf U
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash
bash
bash
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform plan 2>&1 | head -100
},
]
+ tags = {
+ "Name" = "dev-rt"
}
+ tags_all = {
+ "Name" = "dev-rt"
}
+ vpc_id = (known after apply)
}
# aws_default_security_group.myapp_default_sg will be created
+ resource "aws_default_security_group" "myapp_default_sg" {
+ arn = (known after apply)
+ description = (known after apply)
+ egress = [
+ {
+ cidr_blocks = [
+ "0.0.0.0/0",
]
+ from_port = 0
+ ipv6_cidr_blocks = []
+ prefix_list_ids = []
+ protocol = "-1"
+ security_groups = []
+ self = false
+ to_port = 0
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US


Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json U
README.md U
.terraform
.terraform.lock.hcl U
apply_output.log U
entry-script.sh U
main.tf
outputs.tf U
README.md
terraform.tfstate U
terraform.tfvars U
variables.tf U
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash
bash
bash
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform plan 2>&1 | head -100
@hajrasarwar11 →/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform apply -auto-approve 2>&1 | tee apply_output.log
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
# aws_default_route_table.myapp_default_rt will be created
+ resource "aws_default_route_table" "myapp_default_rt" {
+ arn = (known after apply)
+ default_route_table_id = (known after apply)
+ id = (known after apply)
+ owner_id = (known after apply)
+ route = [
+ {
+ cidr_block = "0.0.0.0/0"
+ gateway_id = (known after apply)
# (10 unchanged attributes hidden)
},
]
+ tags = {
+ "Name" = "dev-rt"
}
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US


Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json
README.md
.terraform
.terraform.lock.hcl
apply_output.log
entry-script.sh
main.tf
outputs.tf
README.md
terraform.tfstate
terraform.tfvars
variables.tf
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
bash
bash
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform apply -auto-approve 2>&1 | tee apply_output.log
# aws_instance.myapp_instance will be created
+ resource "aws_instance" "myapp_instance" {
+ ami = "ami-0c94855ba95c574c8"
+ arn = (known after apply)
+ associate_public_ip_address = true
+ availability_zone = "me-central-1a"
+ cpu_core_count = (known after apply)
+ cpu_threads_per_core = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
+ get_password_data = false
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle = (known after apply)
+ instance_state = (known after apply)
+ instance_type = "t3.micro"
+ ipv6_address_count = (known after apply)
+ ipv6_addresses = (known after apply)
+ key_name = "serverkey"
+ monitoring = (known after apply)
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US


Lab_Exam [Codespaces: fictional succotash]
EXPLORER
LAB_EXAM [CODESPACES: ...
.devcontainer
devcontainer.json
README.md
.terraform
.terraform.lock.hcl
apply_output.log
entry-script.sh
main.tf
outputs.tf
README.md
terraform.tfstate
terraform.tfvars
variables.tf
OUTLINE
TIMELINE
APPLICATION BUILDER
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
bash
bash
bash
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $ cd /workspaces/Lab_Exam && terraform apply -auto-approve 2>&1 | tee apply_output.log
+ tags = {
+ "Name" = "dev-ec2-instance"
}
+ tags_all = {
+ "Name" = "dev-ec2-instance"
}
+ tenancy = (known after apply)
+ user_data = "bd5eb80b44c8723a40f689088401474e3cbbbbc3"
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = (known after apply)
+ capacity_reservation_specification (known after apply)
+ cpu_options (known after apply)
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
Codespaces: fictional succotash
main*
AWS: profile:default
Ln 3, Col 1
Spaces: 4
UTF-8
LF
terraform-vars
Layout: US




```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  aws ec2 describe-images --filters "Name=name,Values=al2023-ami-*" --query 'Images[0].ImageId' --region me-central-1 2>&1 | head -20
"ami-003651a04a3c417b7"
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  cd /workspaces/Lab_Exam && terraform destroy -auto-approve 2>&1 | tail -20

Plan: 0 to add, 0 to change, 6 to destroy.

Changes to Outputs:
  - subnet_id = "subnet-0c99526840c03359a" -> null
  - vpc_id    = "vpc-01cf974ef27bc55c9" -> null
aws_key_pair.serverkey: Destroying... [id=serverkey]
aws_default_route_table.myapp_default_rt: Destroying... [id=rtb-02d860751b238f9d6]
aws_subnet.myapp_subnet: Destroying... [id=subnet-0c99526840c03359a]
aws_default_security_group.myapp_default_sg: Destroying... [id=sg-05a3333a060582d20]
aws_default_route_table.myapp_default_rt: Destruction complete after 0s
aws_default_security_group.myapp_default_sg: Destruction complete after 0s
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0c92adefb66f0d77b]
aws_key_pair.serverkey: Destruction complete after 0s
aws_internet_gateway.myapp_igw: Destruction complete after 1s
aws_subnet.myapp_subnet: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-01cf974ef27bc55c9]
aws_vpc.myapp_vpc: Destruction complete after 0s

Destroy complete! Resources: 6 destroyed.
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  cd /workspaces/Lab_Exam && terraform apply -auto-approve 2>&1 | tee apply_o
utput.log
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_default_route_table.myapp_default_rt will be created
  + resource "aws_default_route_table" "myapp_default_rt" {
      + arn                    = (known after apply)
      + default_route_table_id = (known after apply)
      + id                     = (known after apply)
      + owner_id               = (known after apply)
      + route                  = [
          + {
              + cidr_block                = "0.0.0.0/0"
              + gateway_id                = (known after apply)
                # (10 unchanged attributes hidden)
            },
        ]
      + tags                   = {
          + "Name" = "dev-rt"
        }
```

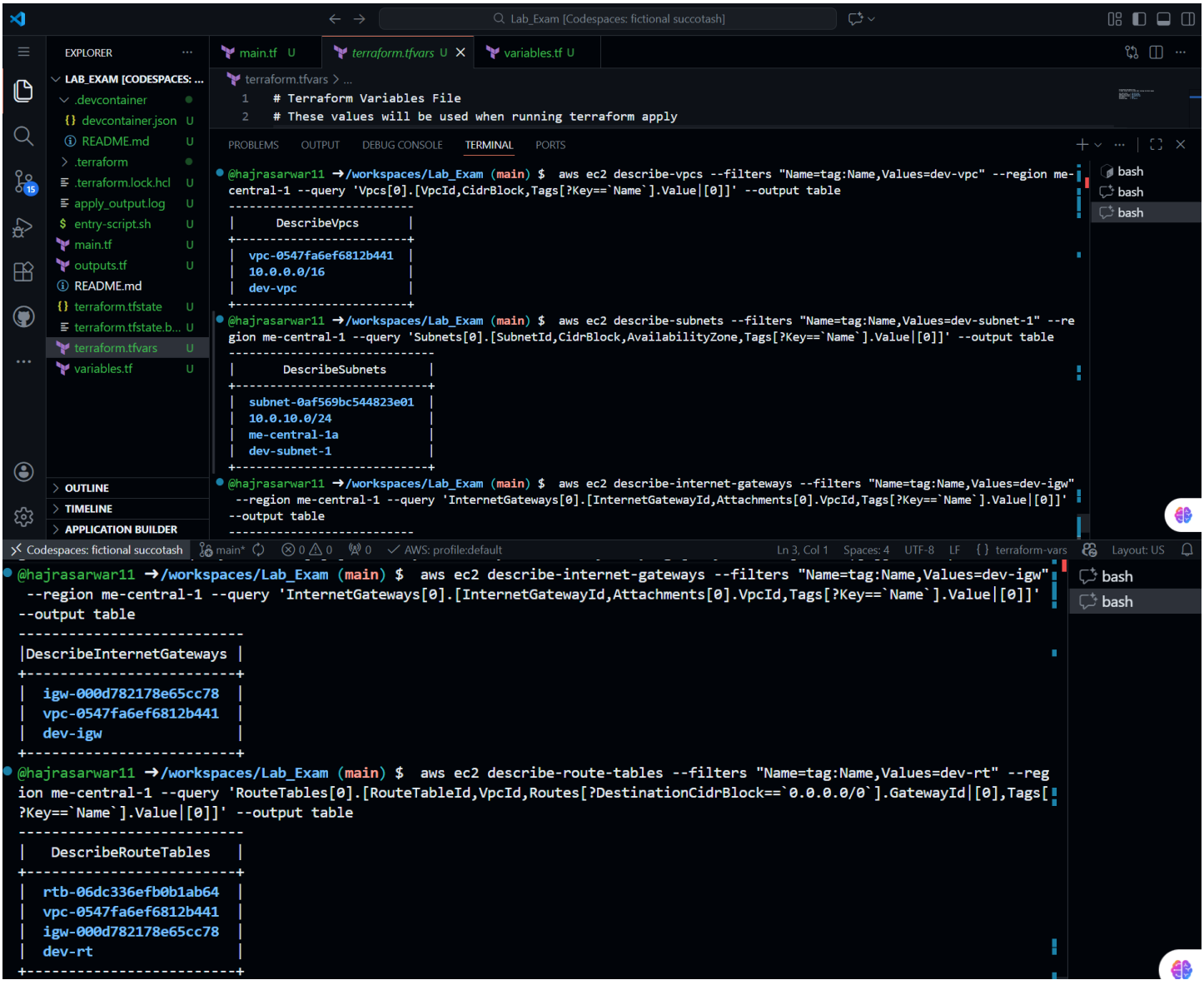
bash
bash

main* ⊗ 0 ⚠ 0 0 ✓ AWS: profile:default Ln 3, Col 1 Spaces: 4 UTF-8 LF { } terraform-vars Layout: US

```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  aws ec2 describe-images --filters "Name=name,Values=al2023-ami-*" "Name=arc
hitecture,Values=x86_64" --query 'Images[0].ImageId' --region me-central-1
"ami-003e2aaab8af13bc1"
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

```
aws_subnet.myapp_subnet: Creating...
aws_default_security_group.myapp_default_sg: Creating...
aws_internet_gateway.myapp_igw: Creation complete after 1s [id=igw-000d782178e65cc78]
aws_default_route_table.myapp_default_rt: Creating...
aws_subnet.myapp_subnet: Creation complete after 1s [id=subnet-0af569bc544823e01]
aws_default_route_table.myapp_default_rt: Creation complete after 2s [id=rtb-06dc336efb0b1ab64]
aws_default_security_group.myapp_default_sg: Creation complete after 4s [id=sg-07a15135d766312de]
aws_instance.myapp_instance: Creating...
aws_instance.myapp_instance: Still creating... [00m10s elapsed]
aws_instance.myapp_instance: Creation complete after 14s [id=i-0e333856f3172e6e5]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

ec2_public_ip = "51.112.52.191"
instance_id = "i-0e333856f3172e6e5"
subnet_id = "subnet-0af569bc544823e01"
vpc_id = "vpc-0547fa6ef6812b441"
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  cd /workspaces/Lab_Exam && terraform output
ec2_public_ip = "51.112.52.191"
instance_id = "i-0e333856f3172e6e5"
subnet_id = "subnet-0af569bc544823e01"
vpc_id = "vpc-0547fa6ef6812b441"
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $
```

## 13. Verift terraform resourses:




```
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  aws ec2 describe-internet-gateways --filters "Name=tag:Name,Values=dev-igw" --region me-central-1 --query 'InternetGateways[0].[InternetGatewayId,Attachments[0].VpcId,Tags[?Key==`Name`].Value|[0]]' --output table
-------------------------
|DescribeInternetGateways |
+-------------------------+
|  igw-000d782178e65cc78  |
|  vpc-0547fa6ef6812b441  |
|  dev-igw                |
+-------------------------+
@hajrasarwar11 ➜/workspaces/Lab_Exam (main) $  aws ec2 describe-route-tables --filters "Name=tag:Name,Values=dev-rt" --region me-central-1 --query 'RouteTables[0].[RouteTableId,VpcId,Routes[?DestinationCidrBlock==`0.0.0.0/0`].GatewayId|[0],Tags[?Key==`Name`].Value|[0]]' --output table
-------------------------
|   DescribeRouteTables   |
+-------------------------+
|  rtb-06dc336efb0b1ab64  |
|  vpc-0547fa6ef6812b441  |
|  igw-000d782178e65cc78  |
|  dev-rt                 |
+-------------------------+
```

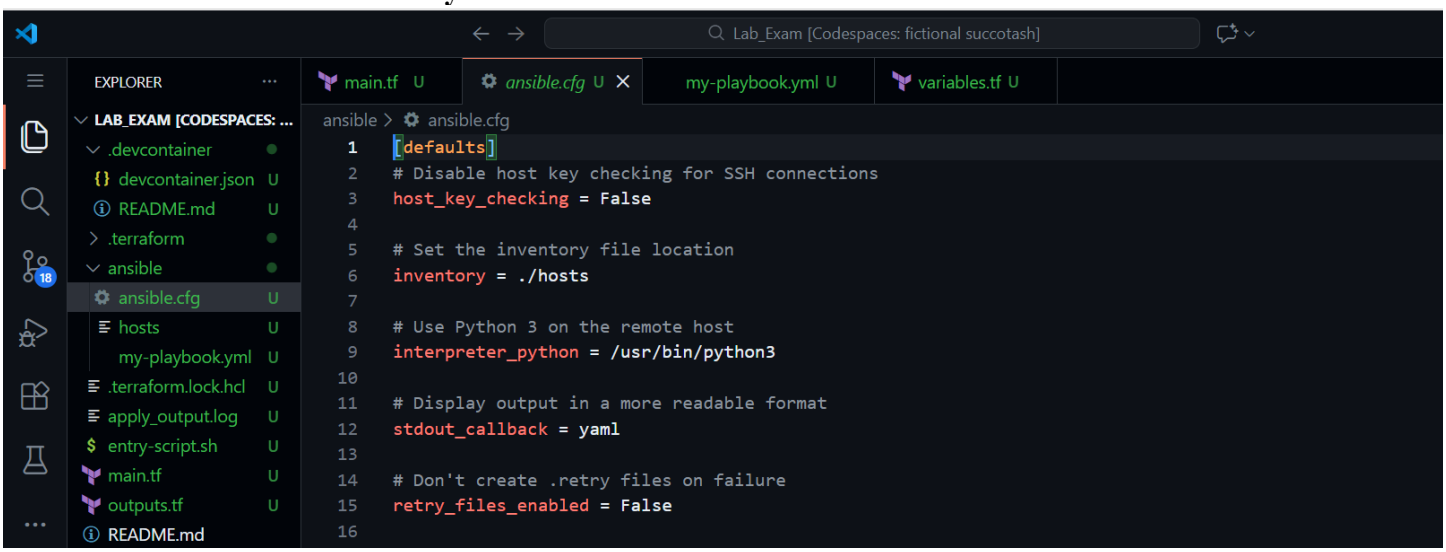
## 14. Verify http:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

@hajrasarwar11 →/workspaces/Lab_Exam (main) $  echo "EC2 Public IP: 51.112.52.191"
EC2 Public IP: 51.112.52.191
@hajrasarwar11 →/workspaces/Lab_Exam (main) $  curl -k -s https://51.112.52.191 2>&1 | head -30
<!DOCTYPE html>
<html>
<head>
    <title>Terraform Environment</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 50px;
            background-color: #f5f5f5;
        }
        .container {
            background-color: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }
        h1 {
            color: #333;
        }
        .info {

bash
bash
bash

main*  0 0  0  AWS: profile:default  Ln 3, Col 1  Spaces: 4  UTF-8  LF  { } terraform-vars  Layout: US

@hajrasarwar11 →/workspaces/Lab_Exam (main) $  curl -k -s https://51.112.52.191 2>&1 | grep -E "(Hajra|Terraform)" | head
-10
    <title>Terraform Environment</title>
        <h1>Welcome to Terraform Environment</h1>
        <p>This is Hajra Sarwar's Terraform environment.</p>
            <p><strong>Environment:</strong> Powered by Terraform</p>
            This instance was provisioned using Terraform with Nginx, SSL/TLS, and automated deployment.
```

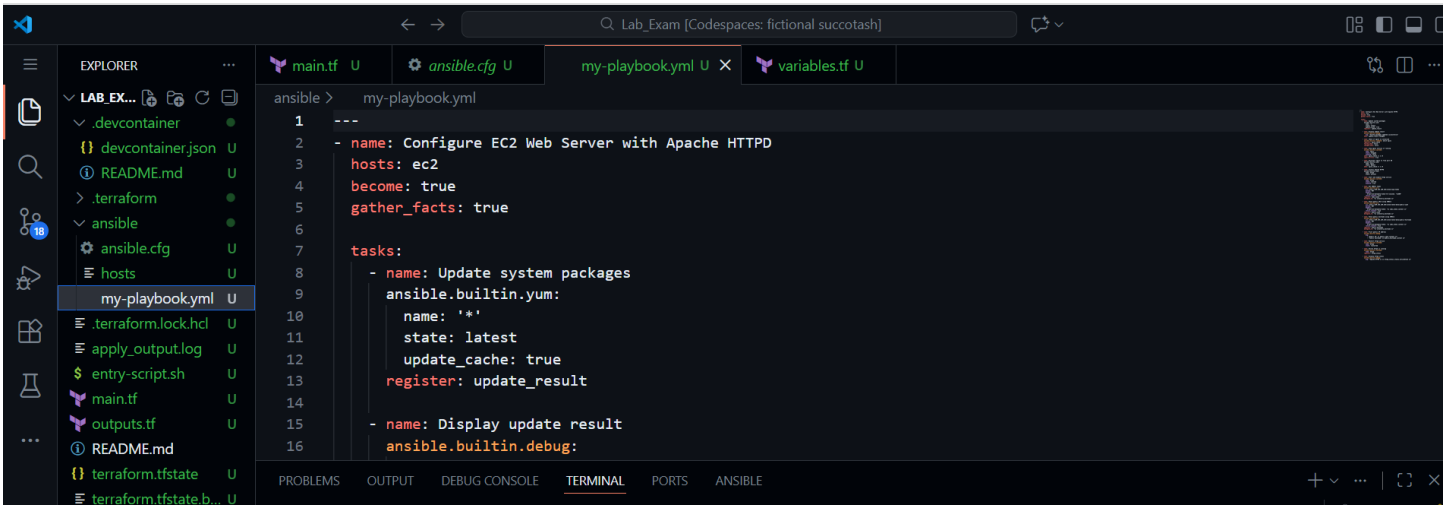
```
@hajrasarwar11 →/workspaces/Lab_Exam (main) $  ssh -o StrictHostKeyChecking=no -i ~/.ssh/id_ed25519 ec2-user@51.112.52.191
 "sudo systemctl status nginx" 2>&1 | head -20
    Process: 2317 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Process: 2318 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
    Process: 2319 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 2320 (nginx)
      Tasks: 3 (limit: 1059)
     Memory: 4.0M
        CPU: 65ms
     CGroup: /system.slice/nginx.service
             ├─2320 "nginx: master process /usr/sbin/nginx"
             ├─2321 "nginx: worker process"
             └─2322 "nginx: worker process"

Jan 19 08:49:26 ip-10-0-10-78.me-central-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse
 proxy server...
Jan 19 08:49:26 ip-10-0-10-78.me-central-1.compute.internal nginx[2318]: nginx: [warn] could not build optimal types_hash,
you should increase either types_hash_max_size: 2048 or types_hash_bucket_size: 64; ignoring types_hash_bucket_size
Jan 19 08:49:26 ip-10-0-10-78.me-central-1.compute.internal nginx[2318]: nginx: the configuration file /etc/nginx/nginx.con
f syntax is ok
Jan 19 08:49:26 ip-10-0-10-78.me-central-1.compute.internal nginx[2318]: nginx: configuration file /etc/nginx/nginx.conf te
st is successful
@hajrasarwar11 →/workspaces/Lab_Exam (main) $

bash
bash
bash
```

## QUESTION 3:

### 1 and 2: Create ensible enventory:




### 3. Create ansible playbook:




### 4. Run notebook:

## 5. Verify HTTP access:

```
@hajrasarwar11 ➜ /workspaces/Lab_Exam/ansible (main) $  timeout 5 curl http://51.112.52.191 2>&1 | head -20
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1802  100  1802    0     0   8352      0 --:--:-- --:--:-- --:--:--  8342
<!DOCTYPE html>
<html>
<head>
    <title>Terraform Environment</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 50px;
            background-color: #f5f5f5;
        }
        .container {
            background-color: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }
```